

CORS Configuration in Spring Boot

CORS = Cross-Origin Resource Sharing.

It is a browser security mechanism that controls whether a web application from **one origin** is allowed to call an API hosted on **another origin**.

For your interview, know the problem, the browser behavior, the Spring Boot configuration, and the security implications.

1. What is an Origin?

An origin is:

```
scheme + host + port
```

For example:

```
Frontend:  
https://myapp.com  
  
Backend:  
https://api.myapp.com
```

These are **different origins** because the hosts differ.

Another example:

```
http://localhost:3000  
http://localhost:8080
```

Different ports → different origins.

2. Why do we need CORS?

Suppose your frontend is:

```
http://localhost:3000
```

and your Spring Boot API is:

```
http://localhost:8080
```

The browser sees:

```
localhost:3000  
↓  
different origin  
↓  
localhost:8080
```

The browser may block the request unless the backend says:

Requests from `http://localhost:3000` are allowed.

The backend does this using CORS response headers.

For example:

```
Access-Control-Allow-Origin: http://localhost:3000
```

3. Important distinction

CORS is primarily a **browser-enforced mechanism**.

A backend-to-backend call such as:

```
BPS → Payment Service
```

doesn't normally get blocked by the browser's CORS policy because there is no browser enforcing it.

This is a very common interview question.

Does CORS protect your backend API from another server calling it?

No. CORS primarily controls browser-originated requests.

You still need proper:

- authentication
- authorization
- network security
- API gateway controls
- rate limiting
- input validation

4. Simple Spring Boot CORS Configuration

You can configure CORS using `WebMvcConfigurer`.

```
@Configuration
public class CorsConfig implements WebMvcConfigurer {

    @Override
    public void addCorsMappings(CorsRegistry registry) {

        registry.addMapping("/api/**")
            .allowedOrigins("http://localhost:3000")
            .allowedMethods("GET", "POST", "PUT", "PATCH", "DELETE")
            .allowedHeaders("*");
    }
}
```

This means:

```
/api/**  
↓  
allow requests from  
http://localhost:3000
```

with the specified HTTP methods.

5. Allow Credentials

Suppose your frontend sends cookies or other browser credentials.

You may configure:

```
registry.addMapping("/api/**")  
    .allowedOrigins("https://myapp.com")  
    .allowedMethods("GET", "POST", "PUT", "DELETE")  
    .allowedHeaders("*")  
    .allowCredentials(true);
```

Important

Don't combine:

```
.allowedOrigins("*")  
.allowCredentials(true)
```

as a general solution.

Wildcard origin and credentialed cross-origin requests have restrictions, and browsers will reject an inappropriate configuration.

For a production payment application, explicitly allow trusted origins.

6. Preflight Request

This is a very important interview topic.

Before certain cross-origin requests, the browser sends an **OPTIONS** request first.

Example:

```
OPTIONS /api/payments  
Origin: https://frontend.example.com  
Access-Control-Request-Method: POST  
Access-Control-Request-Headers: Authorization, Content-Type
```

This is called a **preflight request**.

The server responds with something like:

```
Access-Control-Allow-Origin: https://frontend.example.com  
Access-Control-Allow-Methods: POST, GET  
Access-Control-Allow-Headers: Authorization, Content-Type
```

Then the browser decides whether the actual request is allowed.

7. Why does preflight happen?

Browsers perform preflight for requests that aren't considered "simple" under the CORS rules—for example, certain methods or custom headers.

A very common example is:

```
Authorization: Bearer <JWT>
```

combined with:

```
Content-Type: application/json
```

The browser may send:

```
OPTIONS
↓
Server approves CORS
↓
POST /payments
```

8. CORS with Spring Security

This is important for your Spring Boot interviews.

If your application uses **Spring Security**, configuring CORS only through MVC may not be sufficient depending on how the security filter chain is configured.

Typical configuration:

```
@Bean
SecurityFilterChain securityFilterChain(HttpSecurity http)
    throws Exception {

    http
        .cors(Customizer.withDefaults())
        .csrf(csrf -> csrf.disable());

    return http.build();
}
```

Then define a `CorsConfigurationSource`:

```
@Bean
CorsConfigurationSource corsConfigurationSource() {

    CorsConfiguration configuration = new CorsConfiguration();

    configuration.setAllowedOrigins(
        List.of("https://frontend.example.com")
    );

    configuration.setAllowedMethods(
```

```

        List.of("GET", "POST", "PUT", "PATCH", "DELETE", "OPTIONS")
    );

    configuration.setAllowedHeaders(
        List.of("Authorization", "Content-Type")
    );

    configuration.setAllowCredentials(true);

    UrlBasedCorsConfigurationSource source =
        new UrlBasedCorsConfigurationSource();

    source.registerCorsConfiguration("/**", configuration);

    return source;
}

```

The exact Spring Security API varies with Spring Security version, so in an interview explain the **concept** as well as the code.

9. @CrossOrigin

You can also configure CORS on an individual controller or endpoint.

```

@RestController
@RequestMapping("/payments")
@CrossOrigin(origins = "https://frontend.example.com")
public class PaymentController {

    @GetMapping("/{id}")
    public PaymentResponse getPayment(
        @PathVariable String id) {
        ...
    }
}

```

Or:

```

@CrossOrigin(
    origins = "https://frontend.example.com",
    methods = {RequestMethod.GET, RequestMethod.POST}
)

```

This is useful for specific endpoints, but centralized configuration is usually easier to manage consistently in larger applications.

10. Global vs Controller-Level CORS

Controller level

```

@CrossOrigin(origins = "https://frontend.example.com")

```

Good when only certain controllers need special rules.

Global

```
registry.addMapping("/api/**")
```

Better when you want consistent policy across your APIs.

11. Important CORS Headers

Know these.

Access-Control-Allow-Origin

Specifies which origin is allowed.

```
Access-Control-Allow-Origin: https://myapp.com
```

Access-Control-Allow-Methods

```
Access-Control-Allow-Methods: GET, POST, PUT
```

Access-Control-Allow-Headers

```
Access-Control-Allow-Headers: Authorization, Content-Type
```

Access-Control-Allow-Credentials

Controls whether credentials can be included in cross-origin requests.

Access-Control-Expose-Headers

Allows browser JavaScript to access certain response headers that aren't normally exposed.

12. CORS vs CSRF

Interviewers often ask this immediately.

CORS

Controls:

Which browser origins can make/read cross-origin requests?

CSRF

Protects against:

A malicious website causing a user's browser to send an unwanted authenticated request to your application.

They solve different problems.

For a stateless JWT API using an `Authorization` header, CSRF considerations differ from a traditional cookie/session-based application.

Don't say:

“CORS prevents CSRF.”

That's incorrect.

13. CORS vs Authentication

Another important distinction:

```
CORS
→ Browser-origin policy

Authentication
→ Who is the caller?

Authorization
→ What is the caller allowed to do?
```

For your BPS application:

```
Frontend
↓
CORS
↓
Spring Security
↓
Authentication
↓
Authorization
↓
Payment API
```

CORS doesn't replace authentication.

14. Common production mistake

Avoid:

```
.allowedOrigins("*")
```

just to make CORS errors disappear.

This effectively permits requests from any origin under the applicable CORS rules.

For a payment system, you should configure trusted origins explicitly, for example:

```
.allowedOrigins(
    "https://payments.company.com",
    "https://operations.company.com"
)
```

Development can have:

```
http://localhost:3000
```

while production uses actual trusted frontend origins.

15. Environment-specific CORS

A good design is to externalize allowed origins.

For example:

```
app:
  cors:
    allowed-origins:
      - https://frontend.example.com
```

Then:

```
DEV
→ http://localhost:3000

TEST
→ https://test-frontend.company.com

PROD
→ https://frontend.company.com
```

This avoids hardcoding production URLs into code.

16. Typical interview scenario

Interviewer:

“Your React application is running on port 3000 and your Spring Boot application is running on port 8080. The browser says `Access-Control-Allow-Origin` is missing. What would you check?”

A good answer:

First I'd verify that the frontend and backend are different origins. Then I'd check the backend CORS configuration for the frontend origin and HTTP method. I'd also check whether Spring Security is handling the request, because the CORS configuration must be integrated with the security filter chain. If the browser is sending a preflight `OPTIONS` request, I'd verify that it is permitted and that the response contains the required CORS headers.

That's much stronger than:

“I'll add `@CrossOrigin`.”

17. Payment application example

For BPS, imagine:

```
Operations UI
https://ops.company.com
    ↓
    BPS REST API
https://bps-api.company.com
```

You could allow:

```
.allowedOrigins("https://ops.company.com")
```

but reject:

```
https://random-site.com
```

This means the browser will only permit JavaScript running from the trusted frontend origin to make/read the cross-origin API response according to the CORS policy.

18. Questions you should prepare for interview

Basic

1. What is CORS?
2. Why is CORS required?
3. What is an origin?
4. What is a preflight request?
5. Which HTTP method is used for preflight?
6. What is `Access-Control-Allow-Origin`?

Intermediate

7. How do you configure CORS in Spring Boot?
8. How do you configure CORS globally?
9. How do you use `@CrossOrigin`?
10. How does CORS work with Spring Security?
11. Why is `OPTIONS` important?
12. What happens when CORS validation fails?
13. How do you allow credentials?
14. Why shouldn't you use wildcard origins carelessly?

Cross-questions

15. Is CORS a backend security mechanism?
16. Does CORS affect service-to-service communication?
17. CORS vs CSRF?
18. CORS vs authentication?
19. What causes a preflight request?
20. How would you troubleshoot a CORS error in production?

Interview-ready answer

If they ask:

■ **“Explain CORS and how you configured it in your Spring Boot application.”**

You can say:

“CORS, or Cross-Origin Resource Sharing, is a browser security mechanism that controls whether a frontend from one origin can access APIs hosted on another origin. In our Spring Boot application, we configured allowed origins, HTTP methods and headers, and integrated CORS with Spring Security. For non-simple cross-origin requests, the browser sends an OPTIONS preflight request, so we also ensure that the preflight request is handled correctly. In production we avoid allowing all origins and configure only trusted frontend origins.”

For a **3-year developer**, also understand **Spring Security filter chain + OPTIONS preflight + credentials + CORS vs CSRF**. Those are the likely follow-up areas.